

A training label can reach into tomorrow

Daily research note | 22 September 2026 | Experiment 002 (Q002)

Sidiq review status: Not yet reviewed.

Question, hypothesis and motivation

Can chronological splitting leak future information even with point-in-time features? Our hypothesis is yes: a training target can use an outcome measured after evaluation begins. Q001 checked feature arrival; Q002 checks target information endpoints. Today's evidence uses synthetic timestamps and makes no alpha claim.

Beginner explanation

Imagine training just before January 5. A January 2 row looks historical, but its target might be the return through January 5. Another January 3 target might need the January 7 price. Sorting decision dates does not make either answer safe to teach before January 5.

Purging removes these rows and records their identities for audit. We also remove labels ending exactly at evaluation start: their final input has not been established as usable before the first evaluated decision.

Definitions and evaluation protocol

Decision time: the instant a sample's prediction would be made. **Label/target:** the outcome a model learns to predict. **Label end:** the last information timestamp used to form that target. **Horizon:** elapsed time from decision to label end. **Holdout:** observations reserved for evaluation. **Purging:** removing training labels whose information overlaps the held-out period. **Embargo:** a separate exclusion of later training observations near a test period; there is no later training in this past-only implementation.

The fixture contains eight invented decisions, January 1-8, with target information extending to January 10, all at 00:00 UTC. The predeclared evaluation decision window is January 5 inclusive to January 8 exclusive. Compare ordinary chronological selection, a one-row trailing gap and interval purging. Record row identities, retained counts and overlap counts against the same window. No model is fit and no parameters are tuned.

Hand-worked fixture

ID	Decision day	Label end day	Purged-holdout role
A	1	3	Train
B	2	5	Purge: touches start
C	3	7	Purge: crosses start
D	4	4	Train
E	5	6	Evaluation
F	6	8	Evaluation
G	7	10	Evaluation
H	8	9	Exclude: window ended

Mathematics and the endpoint convention

Let d_i be sample i 's decision time and e_i its label end, with $e_i \geq d_i$. Its information interval is closed: $I_i = [d_i, e_i]$. Let the evaluation decision window be $W = [a, b)$, with $b > a$. The left boundary is included; the right boundary is excluded. This avoids assigning a decision at b to both adjoining evaluation windows.

Candidate training: $C = \{i : d_i < a\}$.

Evaluation: $V = \{i : a \leq d_i < b\}$. **Excluded later rows:** $X = \{i : d_i \geq b\}$.

Two closed intervals $[s, t]$ and $[u, v]$ intersect when $\max(s, u) \leq \min(t, v)$. A closed label interval intersects our half-open W when $d_i < b$ and $e_i \geq a$. For candidate training rows, $d_i < a < b$ already holds, so the overlap test simplifies to $e_i \geq a$. Therefore $P = \{i \text{ in } C : e_i \geq a\}$ is purged, and $T = \{i \text{ in } C : e_i < a\}$ is retained. The strict inequality is deliberate: $e_i = a$ is not retained.

Because every training decision precedes a , this condition also prevents overlap with any evaluation label starting at or after a . Evaluation labels can end beyond b : b selects decisions, not the latest outcome time. Results for such labels cannot be scored until their information is available. The splitter does not enforce that later scoring date; the experiment runner must do so.

Numerical example and audit invariant

For illustration only, invent $P(\text{January } 2) = 100$ and $P(\text{January } 5) = 103$. The simple forward-return target is $y = P(\text{end}) / P(\text{start}) - 1 = 103 / 100 - 1 = 0.03$, or 3%. It is a target, not a strategy return. Teaching it to a model before January 5 reveals an endpoint price not yet established as available. The experiment stores only interval timestamps, not these illustrative prices.

Every input position appears exactly once in T , V , P or X . Within each output, original order is preserved. All retained training endpoints satisfy $e_i < a$; all evaluation decisions satisfy $a \leq d_i < b$. An empty T is a valid diagnostic outcome. The downstream model must skip or reject that fold, never refill T using P . An empty V is rejected, because there would be nothing to evaluate.

Implementation and reproducible commands

The immutable `LabelInterval` record validates nonempty IDs, aware timestamps and endpoint order, then converts times to UTC. `purged_holdout` validates a positive evaluation window, unique IDs and nondecreasing decisions; ties allow different assets at the same instant. It scans once and returns original positions in four tuples. Runtime and storage are $O(n)$, including the duplicate-ID set and output indices.

Run `make purging-experiment` for the JSON fixture result and `make check` for the full validation suite. Run `make note NOTE_DATE=2026-09-22` to rebuild this PDF. Source files are `src/quant_research/validation/purging.py`, `tests/test_purging.py` and `scripts/purging_experiment.py`. The durable protocol and result are in `experiments/purging/README.md` and `experiments/purging/results.json`.

Observed results and checks

Rule	Retained training IDs	Retained count	Overlap count
Decision chronology only	A, B, C, D	4	2
One-row trailing gap	A, B, C	3	2
Interval purging	A, D	2	0

The one-row gap removes safe row D while retaining both unsafe rows B and C. Horizons differ, so recency of the decision does not determine recency of the target's last input. An appropriately conservative gap could work under known horizon bounds, but this chosen one-row gap does not. The comparator is our explicit row selection rule, not a claimed execution of scikit-learn's complete multi-fold TimeSeriesSplit.

All 49 tests passed, including 24 new purging and artifact cases. Endpoint tests distinguish one microsecond before, exactly at and one microsecond after evaluation start. Other cases cover crossing horizons, zero-duration labels, empty training/evaluation, invalid windows, duplicate IDs, unsorted decisions, non-datetime/naive inputs, tied panel decisions, equivalent zones and a London daylight-saving fold. A seeded test checks 100 interval fixtures against an independent general intersection expression and verifies that appending valid later rows does not alter training, purging or evaluation membership.

Two independent experiment runs reproduced the committed JSON bytes and all hand-worked identities/counts. Ruff lint and formatting passed for 19 Python files; strict mypy passed for 13 package files. Dependency consistency passed. The first full check found three overlong experiment-description strings; formatting was corrected and the complete check then passed. No coverage percentage was measured or claimed.

Limitations and research interpretation

Zero overlap on this fixture proves neither predictive ability nor statistical independence. Targets within evaluation may overlap each other; uncertainty estimates must still account for that dependence. We do not have empirical market data, a fitted model or a return series. Counts here describe designed examples only.

An endpoint is not automatically an availability timestamp. Vendor delays, revisions and label-processing latency still need auditing. If a label arrives later than `e_i`, that additional constraint must be checked before model fitting. Q001's feature availability logic remains necessary. Training-only preprocessing, universe history, corporate actions, calendars, transaction costs and multiple testing are separate tasks.

This is one fixed-model past-only holdout. It does not generate expanding folds, refit inside evaluation, allow future-side training or implement post-test embargo. It intentionally rejects unsorted decisions instead of silently reordering aligned features and targets. Zero-duration intervals are allowed for general labels; whether they make economic sense depends on the eventual target definition.

UTC conversion prevents repeated local-clock hours from being ordered as if they were the same instant. It does not validate the provenance of a supplied timezone or detect a fabricated local timestamp. The existing Q001 selector's direct comparisons have not been extended here; a follow-on audit is recorded in the backlog for named-zone daylight-saving comparisons before broader timestamp ingestion.

Primary sources and design decisions

Sources consulted on 22 September 2026. These support the concepts and timestamp contract; the fixture, endpoint convention and reported counts are our own explicitly stated design and measured result.

1. scikit-learn, TimeSeriesSplit official API documentation. It describes chronological folds and a gap in sample counts. This motivates checking variable label horizons directly rather than assuming a fixed row count is a sufficient interval audit.

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html

2. MLFinLab, cross-validation implementation documentation. It describes purging training samples with overlapping information and distinguishes embargo. Our implementation uses only the past-side holdout case; we have not implemented its general PurgedKFold or combinatorial procedure.

https://random-docs.readthedocs.io/en/latest/implementations/cross_validation.html

3. Python, datetime official documentation. Aware timestamps identify instants, but comparisons sharing the same timezone object ignore fold. We convert to UTC before ordering and test both occurrences of London's repeated autumn hour. <https://docs.python.org/3/library/datetime.html>

What to explain in an interview

Start with a January 2 sample whose target needs a January 5 price. Explain why an old row date does not make its answer known at training time. Derive $e_i < a$ from the interval intersection rule, state the equality convention and explain the sample-size cost. Describe why a fixed gap can fail with variable horizons. Then state the limitation: eliminating this leakage path does not validate data provenance, model fitting or trading profitability. Describe this repository's implemented evidence without implying personal review.

Next eligible work

Wednesday 23 September uses C001: a first C++ savings-growth calculation, as scheduled in CPP_LEARNING_PLAN.md. Q003, experiment manifests and source checksums, remains the next research item on the following non-C++ day. No personal approval is required to continue. Q003 should preserve synthetic data labels and evaluation protocols in machine-checked metadata. The availability timestamp audit is recorded as Q005a before Q005.

Check your understanding

1. Why is B removed even though its decision is January 2?
2. Why does removing the most recent training row fail in this fixture?
3. Is a label ending one microsecond before evaluation start retained? What happens at exact equality?
4. Does an evaluation label ending after b belong in the evaluation set, and when can it be scored?
5. What should happen when purging removes every training row? Does zero overlap establish alpha?

Suggested answers

1. B's label uses information through January 5, the evaluation start. Closed intervals share that instant, so the conservative pre-evaluation fitting rule purges B even though its decision is older.
2. D ends on January 4 and is safe, whereas older rows B and C end on January 5 and January 7. Decision recency cannot substitute for checking each endpoint when horizons vary.
3. Yes, assuming all other validation and upstream availability conditions hold. At equality it is purged; $\text{end} < \text{start}$ is required, not $\text{end} \leq \text{start}$.
4. Yes, when its decision lies in $[a, b)$. Outcomes may extend beyond the decision window. Score only after all target information is available, including any receipt or processing delay.
5. Report and skip or reject the fold. Do not reinstate removed rows. Zero interval overlap addresses one engineering condition and says nothing by itself about predictive skill, statistical independence or alpha.